

CS683: Advanced Computer Architecture

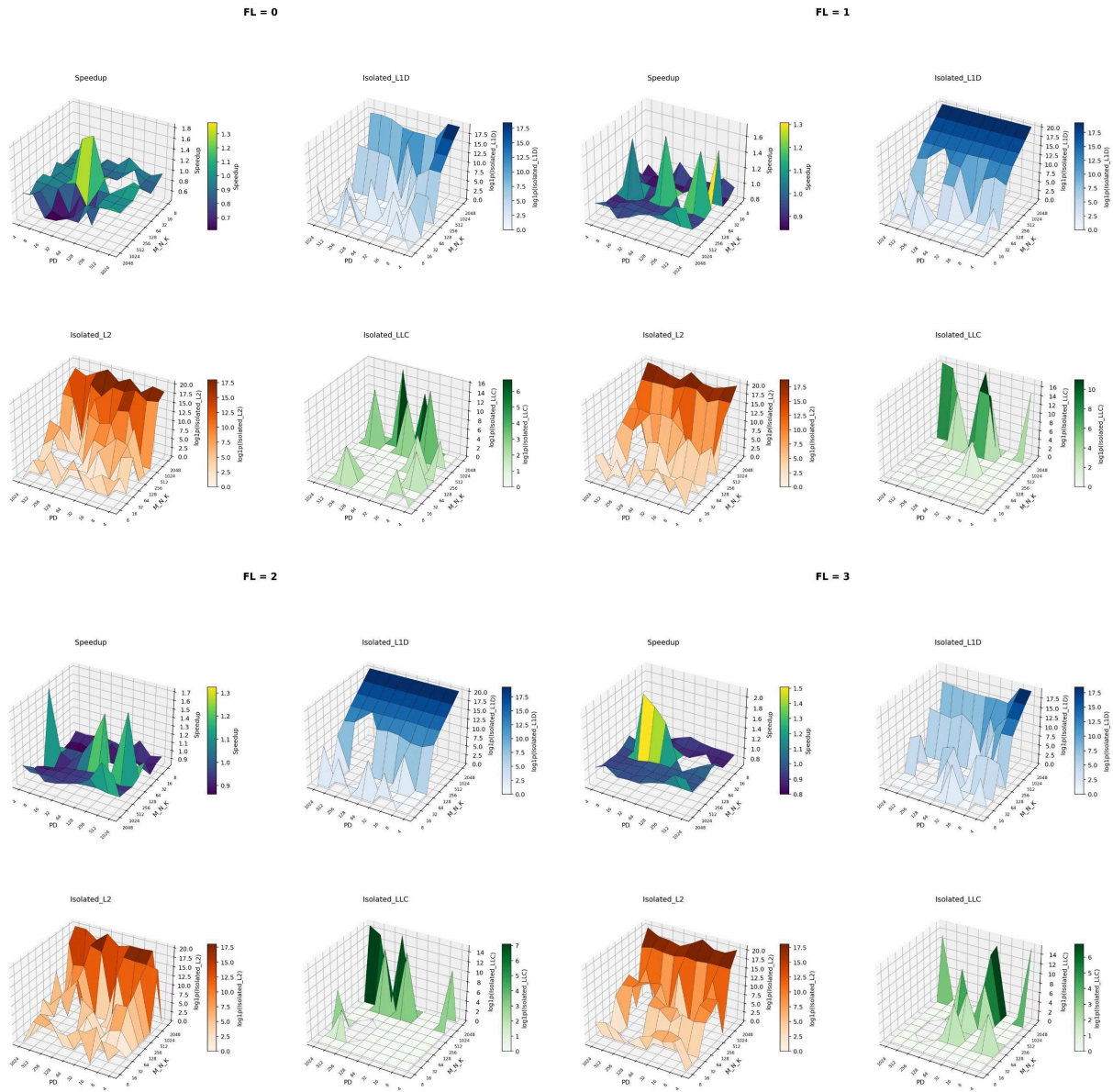
Dhurandhar Programming Assignment 1

Disclaimer: The memes in this document are intended solely for **best report award. They are not meant to offend, mislead, or be taken as factual information. Please try not to cut marks in case you do not enjoy memes.*

Team name : Cache me if you can

L1 Cache Pramod 24b1269	L2 Cache Mukunth 24b1267
L3 Cache Vishal 24b1293	DRAM Arsh 24b1266

Task 2A: Software prefetching



Q1) What trend do you observe in speedup with different matrix sizes?

Small matrices do not seem to obtain much speedup from software prefetching. This might be due to the matrix fitting completely into memory

and there is no point in prefetching. Larger matrices benefit highly from prefetching as shown by the speedup peaks.

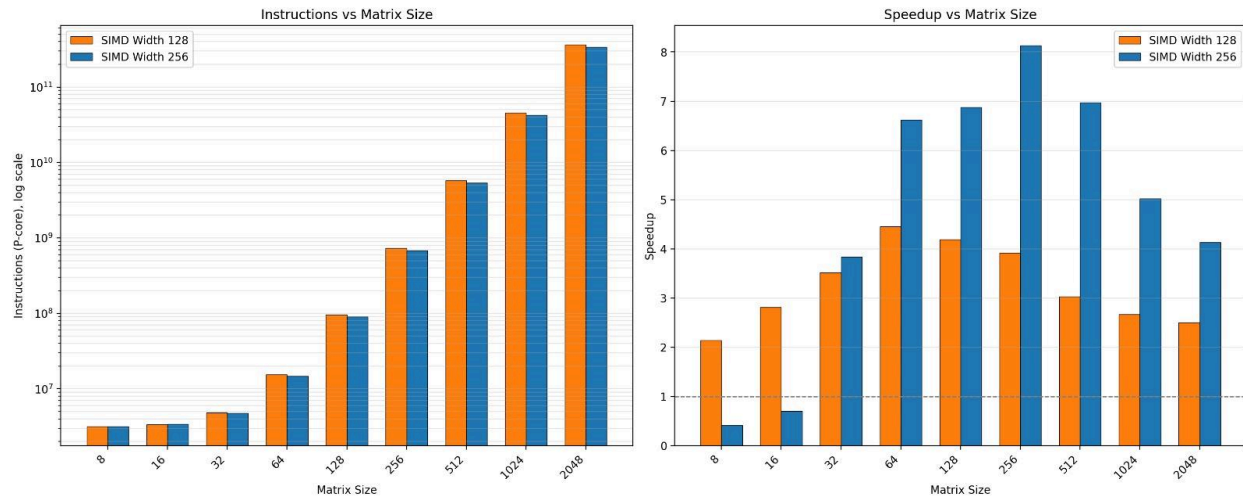
Q2) What is the best prefetch distance?

The best prefetch distance is **16** for cache fill level **FL3**, i.e. all the caches L1, L2 and L3 are filled. Though generally over the different cache fill levels 64 seems to give an overall better performance.

Q3) At what cache fill level do you achieve the maximum speedup.

We achieve maximum speedup for cache fill level 3 where all the 3 caches L1, L2, L3 are filled. The plot shows a very clear global maxima peak.

Task 2B:



Q1) Analyze the Impact of SIMD Instruction Width:

Only 128 width and 256 width simd instructions are available on our machines.

Generally speaking width 256 simd executed faster than 128 simd instructions. The speed is almost 2x , it is less than 2x because of the loop overheads and the rest of the code which cannot be paralalized.

Q2. Analyze the Impact of matrix size:

As matrix size increases 128 wide instructions reach a maxima at 64x64 and 256 wide instructions reach a maxima at 256 and they fall off.

Q3. Analyze Instruction Count:

Instruction count increases with size of the matrix for both 128 wide and 256 wide. They are almost equal for the same matrix size.

Q4. Collect Execution Time and Compute Speedup:

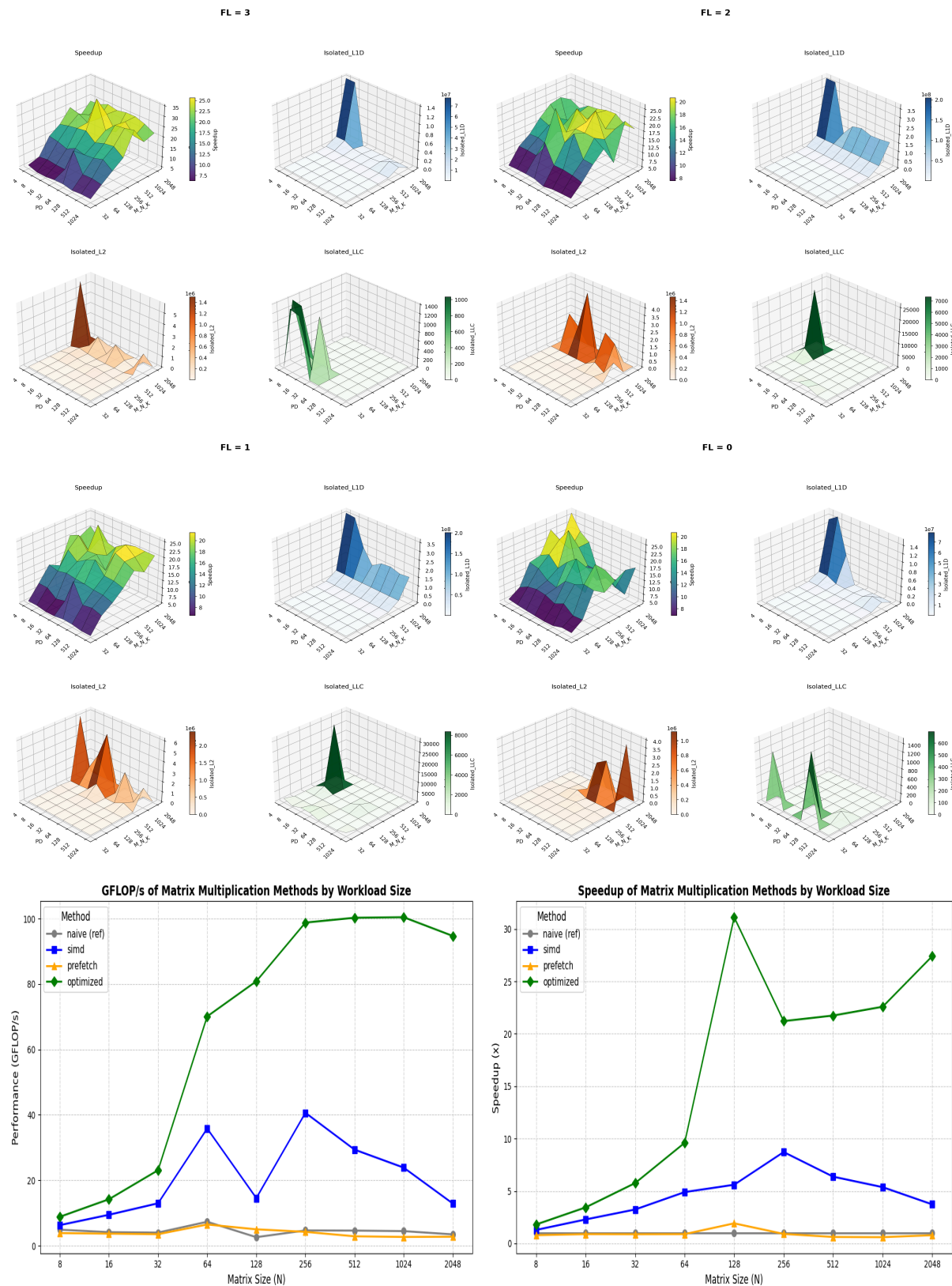
Speedup was the most for 256 wide as expected and its maxima was at matrix size of 256.

Table:

Matrix_Size	Implementation	SIMD_Width	Instructions	Time (ms)	Speedup
8	No SIMD	N/A	3149641	0.000	1.00
8	SIMD	128	3163208	0.000	2.14
8	SIMD	256	3171962	0.001	0.41
16	No SIMD	N/A	3340261	0.002	1.00
16	SIMD	128	3371114	0.001	2.81
16	SIMD	256	3413296	0.002	0.70
32	No SIMD	N/A	4490843	0.010	1.00
32	SIMD	128	4813726	0.003	3.52
32	SIMD	256	4773228	0.002	3.84
64	No SIMD	N/A	13308919	0.082	1.00
64	SIMD	128	15414203	0.020	4.45
64	SIMD	256	14746051	0.013	6.62
128	No SIMD	N/A	81630073	0.758	1.00
128	SIMD	128	96556233	0.189	4.19
128	SIMD	256	90772372	0.110	6.88
256	No SIMD	N/A	619979103	6.618	1.00
256	SIMD	128	731941837	1.760	3.92
256	SIMD	256	682447689	0.815	8.13
512	No SIMD	N/A	4888422851	56.905	1.00
512	SIMD	128	5757175446	18.912	3.03
512	SIMD	256	5348934920	8.121	6.97

```
1024,No SIMD,N/A,38891639391,467.988,1.00
1024,SIMD,128,45726761128,176.320,2.67
1024,SIMD,256,42412719049,92.864,5.02
2048,No SIMD,N/A,310416976939,5075.779,1.00
2048,SIMD,128,364682961849,1964.907,2.50
2048,SIMD,256,337976461967,1188.512,4.13
```

Task 2C: Software Prefetching + SIMD



Q1) What trend do you observe in speedup with different matrix sizes?

Small matrices do not seem to obtain much speedup from software prefetching. This might be due to the matrix fitting completely into memory and there is no point in prefetching. Larger matrices benefit highly from prefetching as shown by the speedup peaks.

Q2) What is the best prefetch distance?

The best prefetch distance is **16** for cache fill level **FL3**, i.e. all the caches L1, L2 and L3 are filled. Though generally over the different cache fill levels 64 seems to give an overall better performance.

Q3) At what cache fill level do you achieve the maximum speedup.

We achieve maximum speedup for cache fill level 3 where all the 3 caches L1, L2, L3 are filled. The plot shows a very clear global maxima peak.

BLOOPERS

For more serious people, skip this section. Kind request not to lower our marks. Thank you for your cooperation.

Code with no optimization:

I was experimenting with software prefetching and needed a complete restart so I deleted some code, some leftover tiling was there and thus I had some 16x speedup...

Code Snippet:

```
for (int i = 0; i < M; ++i) {  
    for (int j = 0; j < N; j+=16) {  
        float acc = 0.0f;  
        const float* a = A + static_cast<long>(i) * lda;  
        const float* b = B + static_cast<long>(j) * ldb;  
        for (int p = 0; p < K; ++p) {  
            acc += a[p] * b[p];  
        }  
        C[static_cast<long>(i) * ldc + j] = acc;  
    }  
}
```

So in an obvious glance the problem was obvious, the correctness was compromised and so I checked the correctness of code... only to find a yes there. So yay I got 16x speedup!!!

**Me telling the team I got a 16x speedup
by tweaking the naive loop**



**The autograder when it checks the Maximum
Absolute Error and sees I just wrote $k+=16$**

*Later I realized the grader only checks if you change the matrix entry and not all entries very efficient if you ask me.

1x	
16x	
64x	
256x	
1024x	

2) Naive Code Achieving Exponential speedups

Reasons:

Quite Obviously, 16x is when +=16
32x when +=32 and so on. The
speedup comes since we skip that
many calculations and correctness is
always **yes**. Glad the professor spoke
about such instances in class.

***Arsh
Intelligence**



How Arsh feels after generating memes